UNIVERSIDADE ESTADUAL DO OESTE DO PARANÁ

CIÊNCIA DA COMPUTAÇÃO

JOÃO PEDRO ROJAS, RAFAEL MENDONÇA VAZ, VINÍCIUS ALMEIDA HERMES

IMPLEMENTAÇÃO DE APLICAÇÃO WEB COM VUE.JS

Trabalho apresentado à disciplina de Tecnologias para Desenvolvimento de Software, ministrada pelo Professor MSc. Leonardo Medeiros

CASCAVEL

2026

SUMÁRIO

1. INTRODUÇÃO..... 3
2. DESCRIÇÃO DAS FUNCIONALIDADES..... 3
3. DESCRIÇÃO DO FLUXO DE DESENVOLVIMENTO..... 4
3.1 PREPARAÇÃO DO AMBIENTE..... 4
3.2 DESENVOLVIMENTO DA FUNCIONALIDADE..... 5
3.3 COMPONENTIZAÇÃO E FRONT-END..... 8
4. CONCLUSÃO..... 10
5. REFERÊNCIAS..... 11

1. INTRODUÇÃO

Neste trabalho foi implementado uma aplicação Web inspirada no jogo “Pergunte ao Polvo”, utilizando os frameworks Vue.js e o Bootstrap para o desenvolvimento de componentes de interface, junto com eles foi utilizado também o ambiente do Node.js para a execução do servidor web. Além disso, foi utilizado o serviço do GitHub através da ferramenta Git para o versionamento e contribuições no código, por parte dos desenvolvedores. Por fim, a aplicação foi hospedada no serviço gratuito da Vercel, para ser acessada por qualquer pessoa.

2. DESCRIÇÃO DAS FUNCIONALIDADES

Nesse projeto, sua finalidade básica é receber como entrada duas strings e após isso realizar uma escolha aleatória entre elas e retornar a escolhida. Em relação à estrutura do trabalho, foi organizado em componentes, separando as partes da interface, processamento e controle.

Sobre a interface, é a parte em que ocorre o contato com o usuário, sua implementação está localizada em uma pasta *components*, que por sua vez está separada em elementos. O elemento *CabecalhoPagina.vue* representa a principal identidade visual da página, contendo a foto do Trinca-Ferro e o título. O componente *FormularioOpcoes.vue*, é a parte do usuário inserir os textos, esse formulário utiliza o *v-model* do Vue, que relaciona as entradas com as variáveis. O componente *TelaInicio.vue* faz a junção dos dois elementos anteriores, estruturando a tela inicial. Após o processamento, a interface é substituida pela tela do arquivo *TelaResultado.vue* que é composta pela página inicial junto com outra tela *ResultadosGerados.vue*, apresentando os textos inseridos e qual foi a escolha do Trinca-Ferro, junto com uma opção de fazer uma nova iteração.

A parte do processamento, representa a lógica implementada para realizar a escolha entre as duas opções de entradas feitas pelo usuário, essa funcionalidade está contida no arquivo *rand.js* dentro da pasta *logic*. Essa escolha é feita de uma maneira muito simples, primeiro é gerado um número decimal aleatório entre 0 a 1 exclusivo, pela função *Math.random()* do Javascript, com esse valor é feito uma comparação, para ver se o número gerado é maior ou menor que 0.49, selecionando a primeira ou segunda opção.

E por fim, a parte de controle está contida no arquivo *App.vue*, que é responsável pelo gerenciamento dos dados e comunicação entre os componentes, assim como, está contido nele as funções responsáveis por receber as strings, executar a lógica de escolha e reiniciar os valores após uma nova tentativa. A exibição das telas, consiste em aparecer a tela principal enquanto não existir um resultado final de escolha, no caso apenas existirá após o processamento e em seguida é realizado a troca de tela. A comunicação entre os componentes são realizadas através de props e $emit, que permite o compartilhamento de informações. O arquivo main.js também faz parte dessa estrutura, no qual, inicializa o Vue e realiza a montagem do componente principal no documento HTML.

3. DESCRIÇÃO DO FLUXO DE DESENVOLVIMENTO

3.1 PREPARAÇÃO DO AMBIENTE

Em primeiro momento, foi realizada a criação do projeto Vue.js através do Vue CLI em sua versão 5.0.9, através do comando "`vue create pergunte-ao-trinca-ferro`". Em seguida, foram selecionadas algumas opções pertinentes à configuração do projeto e seu ambiente, como ilustra a Figura 1, com a informação mais relevante sendo a versão do Vue, a qual foi escolhida a v3. Com isso, a criação do projeto foi bem sucedida, como ilustra a Figura 2.

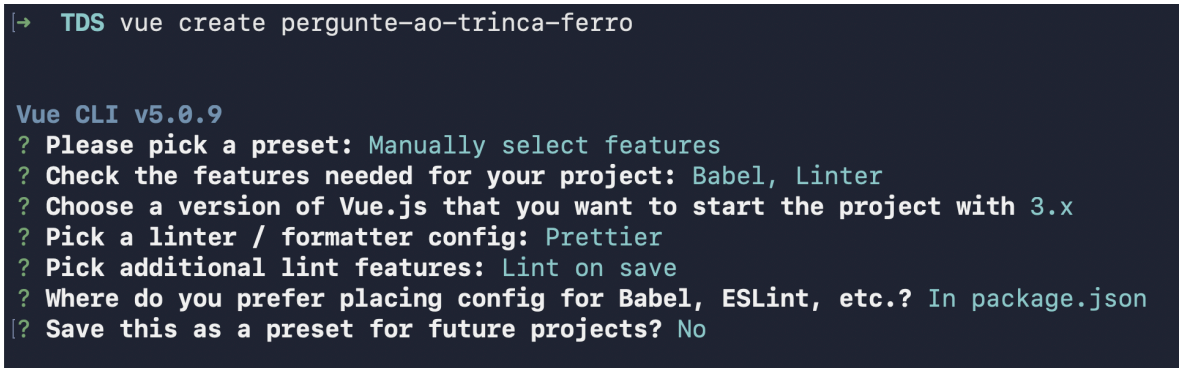


Figura 1. Opções de projeto e ambiente por meio do Vue CLI

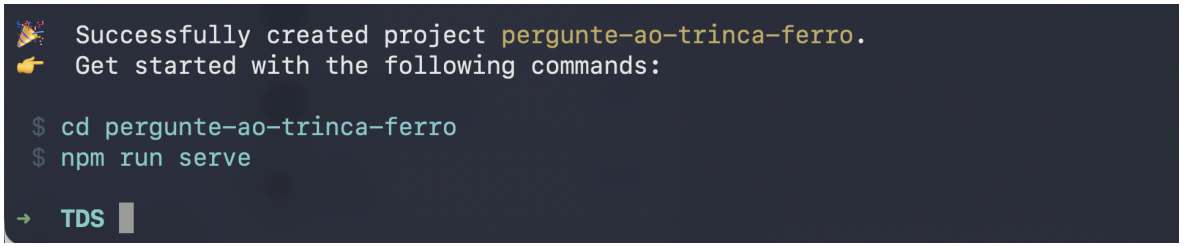


Figura 2. Criação bem sucedida do projeto

Por fim, antes de iniciar o desenvolvimento da aplicação, utilizamos a ferramenta Git para adicionar a um repositório no GitHub o projeto, para todos os desenvolvedores do grupo poderem clonar o repositório e fazer alterações. A Figura 3 ilustra o processo de inicializar um repositório no diretório local do projeto e fazer o primeiro *commit* e *push* para a plataforma. O repositório pode ser consultado através do link https://github.com/vinihrms/pergunte-ao-trinca-ferro. Posteriormente, o repositório também foi hospedado na plataforma open-source Codeberg, como parte do requisito da atividade avaliativa, podendo ser acessada através de https://codeberg.org/vinciush/pergunte-ao-trinca-ferro.

```
→  TDS cd pergunte-ao-trinca-ferro
→  pergunte-ao-trinca-ferro git:(master) git init

Reinitialized existing Git repository in /Users/vncs/CC/2o ano/TDS/pergunte-ao-trinca-ferro/.git/
→  pergunte-ao-trinca-ferro git:(master) git add .
→  pergunte-ao-trinca-ferro git:(master) git commit -m "first commit"

On branch master
nothing to commit, working tree clean
→  pergunte-ao-trinca-ferro git:(master) git branch -M main

→  pergunte-ao-trinca-ferro git:(main) git remote add origin https://github.com/vinihrms/pergunte-ao-trinca-ferro.git

→  pergunte-ao-trinca-ferro git:(main) git push -u origin main

Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 10 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (19/19), 112.24 KiB | 12.47 MiB/s, done.
Total 19 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/vinihrms/pergunte-ao-trinca-ferro.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
→  pergunte-ao-trinca-ferro git:(main)
```

Figura 3. Criação do repositório no GitHub

Agora, com o comando "`npm run serve`", é possível iniciar o servidor local e observar que a criação do ambiente funcionou corretamente, exibindo a página padrão do Vue.js, como ilustra a Figura 4.

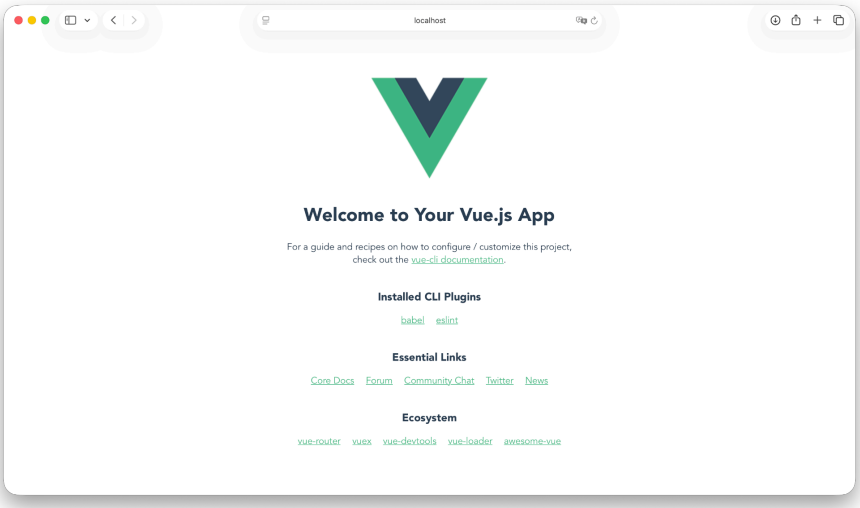


Figura 4. Página inicial padrão do Vue.js

3.2 DESENVOLVIMENTO DA FUNCIONALIDADE

Esta parte foi desenvolvida em uma nova *branch* para não impactar o fluxo de desenvolvimento dos componentes, e poder seguir de forma independente às etapas de componentização.

Seguindo a idealização inicial do projeto, a lógica da funcionalidade envolve uma função seletora, como ilustra a Figura 5, que recebe dois parâmetros “a” e “b”, e retorna um deles, baseado em um valor gerado aleatóriamente. O valor aleatório é gerado pela biblioteca “Math.random()”, inclusa no Node.JS, e gera valores dentro do intervalo [0, 1), e à partir deste valor, a função seletora decide entre o parâmetro “a” ou “b” e o retorna.

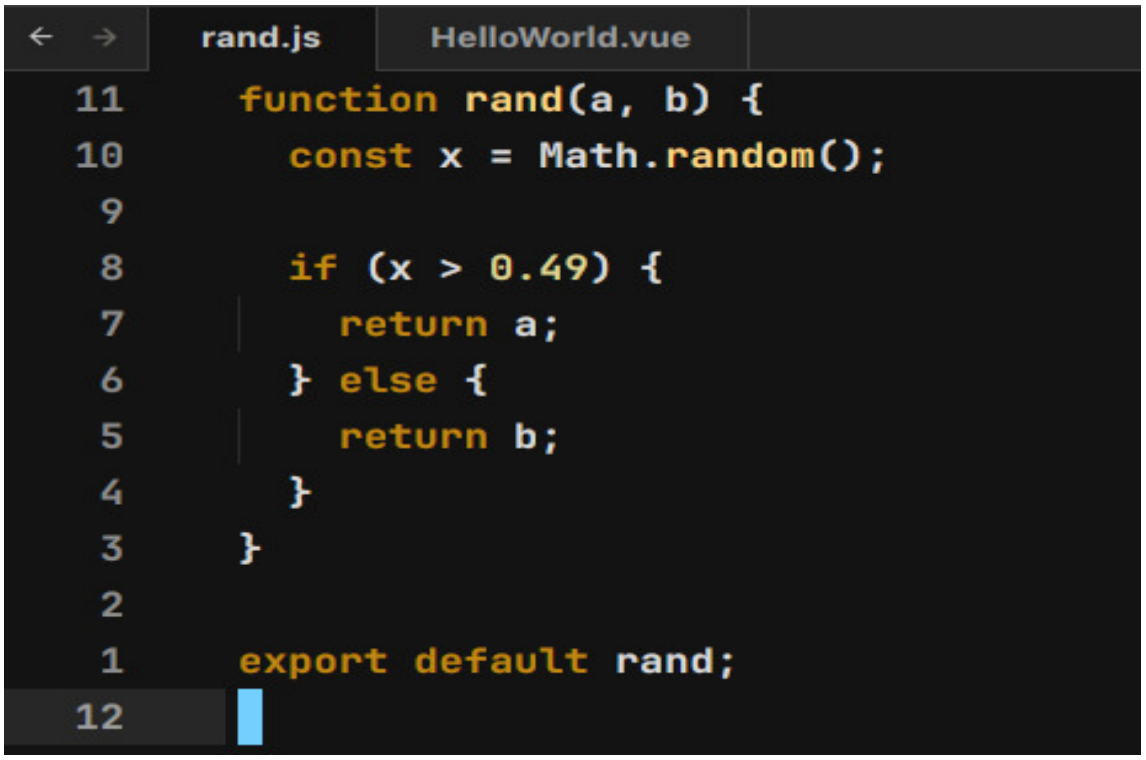


Figura 5. Função principal da aplicação

Para testar se a função iria se portar como esperado, foi criado um pequeno trecho na página de boas vindas do Vue, inserindo dois inputs e um botão, que deveria renderizar a resposta da função. Duas variáveis foram criadas para receber os valores recebidos pelo input, que posteriormente seriam passados como parâmetros da função *rand*. A resposta seria armazenada em outra variável que renderiza em tela o valor retornado pela função.

```
<script setup>
import { ref } from "vue";
import rand from "../logic/rand";

const a = ref("");
const b = ref("");
const res = ref(null);
function run() {
  res.value = rand(a.value, b.value);
}
</script>
```

Figura 6. Área do script de teste no arquivo *HelloWorld.vue*, importando a função e criando as variáveis.

Dentro do *template*, foi inserido os atributos HTML que recebem as opções através dos inputs, e um botão que chamaria o evento de execução da função, junto de uma renderização condicional em uma tag de parágrafo.

```
<div>
  <h1>Testar Função</h1>

  <input v-model="a" placeholder="Valor 1" />
  <input v-model="b" placeholder="Valor 2" />

  <button @click="run">Executar</button>

  <p v-if="resultado != null">Resultado: {{ res }}</p>
</div>
```

Figura 7. Estrutura HTML dentro do template, responsável pelo teste de renderização.

A visualização final do teste ficou conforme a figura abaixo, e o teste de execução obteve sucesso, onde após a inserção de valores dentro dos inputs e o click no botão Executar, a aplicação atualizava a renderização de tela e exibia um dos valores inseridos no input, de acordo com o funcionamento de sua lógica interna.

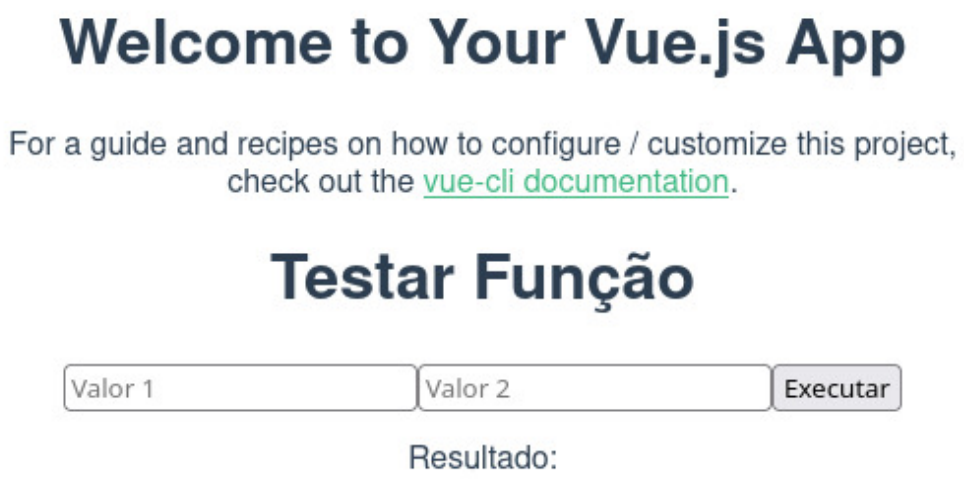


Figura 8. Página de teste da lógica principal da aplicação, exibindo os inputs e renderizando o resultado da função *rand* após clicar no botão Executar.

Com os testes concluídos e a lógica funcionando conforme o esperado, os trabalhos nesta *branch* foram encerrados, aguardando o término da componentização e aspectos estéticos do Frontend, para que pudesse ser realizado o *merge* e junção das duas partes.

3.3 COMPONENTIZAÇÃO E FRONT-END

Para o desenvolvimento do front-end da aplicação, foi utilizado do conceito central dos frameworks SPA (single page application), como o Vue.js, que é a abstração de elementos da página para componentes, permitindo a sua reutilização. Com isso, criamos alguns arquivos de componentes, com o componente do cabeçalho da página sendo reutilizado entre as duas páginas da aplicação (componentes). A primeira página contendo também um formulário com dois campos, para escrever as duas opções, e um botão para enviar as opções à lógica de programação que, logo após a submissão, renderiza o componente responsável pela página dos resultados, que exibe as duas opções digitadas e a que o Trinca-Ferro escolheu, com um botão para limpar os formulários, a resposta e retornar à tela inicial. A Figura 9 ilustra os componentes criados, e a implementação de cada um deles pode ser conferida no repositório do projeto.

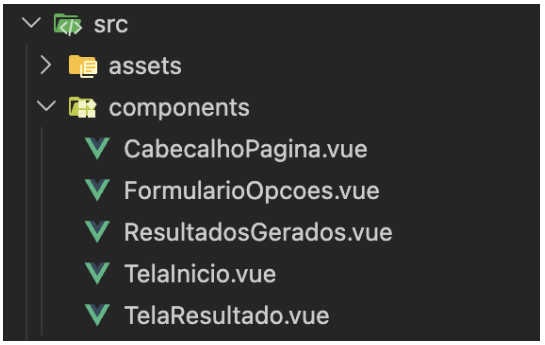


Figura 9. Componentes da aplicação

Ao renderizar a página inicial, os componentes já interligados e estilizados com a biblioteca do Bootstrap 6 sua exibição está ilustrada pela Figura 10.

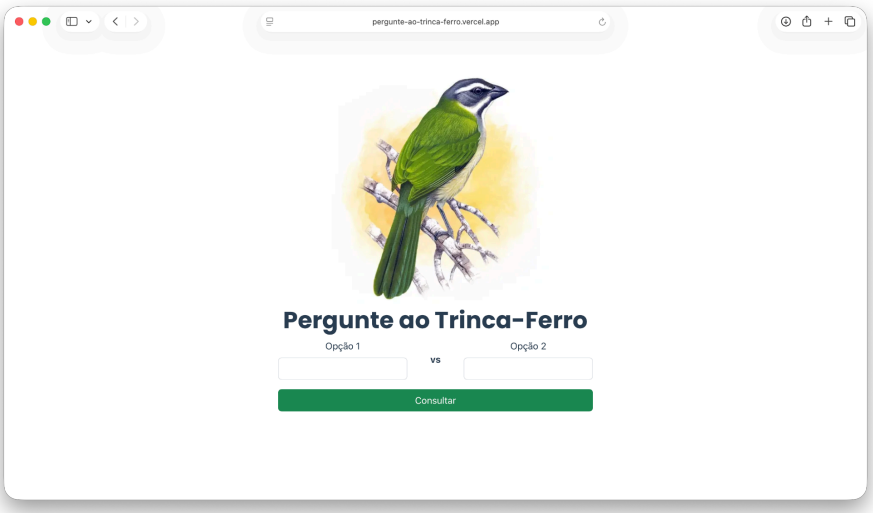


Figura 10. Página inicial da aplicação

Após preencher os dois campos com a opção desejada, pode-se clicar no botão consultar, para realizar a consulta com o Trinca-Ferro, que decidirá através do método explicado anteriormente no presente trabalho, qual opção é a melhor. Em seguida, é renderizada a página exibindo a escolha, como ilustra a Figura 11.

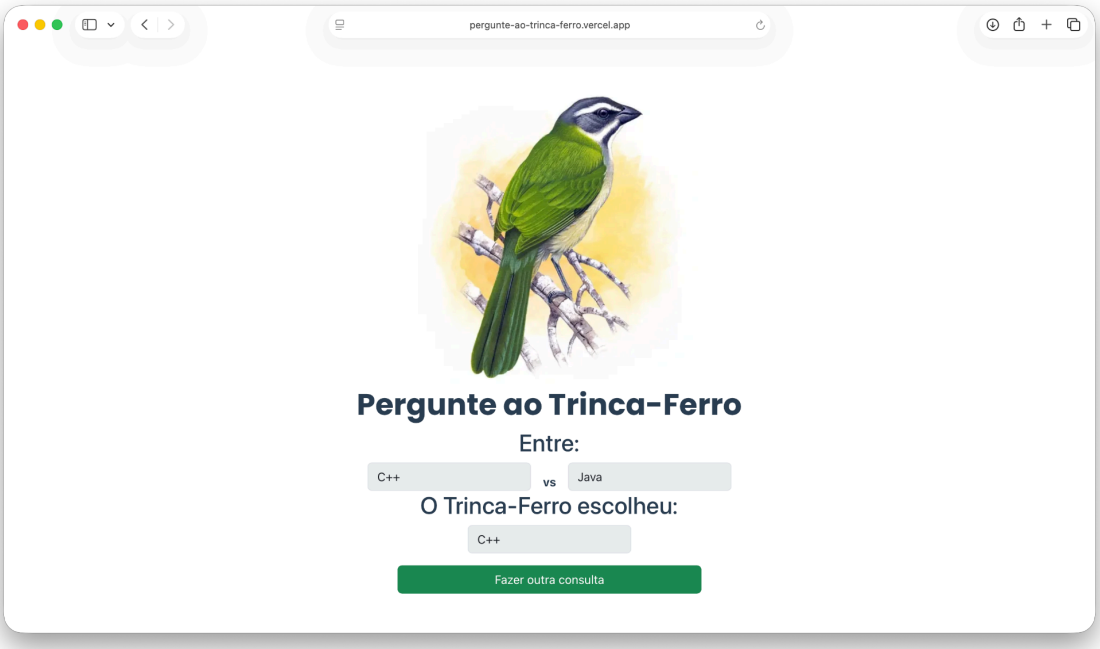


Figura 11. Renderização da escolha do Trinca-Ferro

3.4 HOSPEDAGEM

Em último momento, a aplicação foi hospedada no serviço de hospedagem gratuita Vercel, na qual utiliza do próprio repositório do GitHub para realizar a *build* da aplicação, uma vez que as contas estão vinculadas. O processo foi relativamente simples, apenas sendo necessário escolher o repositório a ser hospedado. Vale lembrar que a plataforma faz o *deployment* automático por meio de cada *commit* de atualização da aplicação. A Figura 12 ilustra o *status* da hospedagem na plataforma Vercel, que pode ser acessada e testada pelo link https://pergunte-ao-trinca-ferro.vercel.app/.

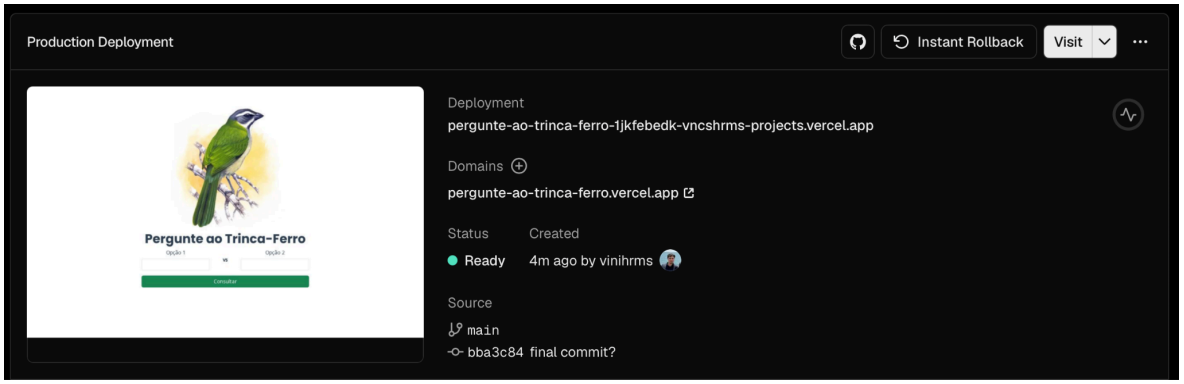


Figura 12. *Status* de hospedagem da aplicação

4. CONCLUSÃO

O desenvolvimento deste trabalho permitiu compreender de forma prática o funcionamento das principais tecnologias utilizadas para criação e gerenciamento de *containers* no sistema operacional *Linux*. Primeiramente, foi realizado a implementação manual utilizando o *RunC*, no qual possibilitou compreender os mecanismos fundamentais empregados pelo kernel Linux, como *namespaces*, *cgroups*, sistema de arquivos (*rootfs*) e arquivos de configuração responsáveis pela execução de *containers*, como o *config.json.*

Em seguida, foram exploradas ferramentas que automatizam alguns passos que foi necessário aprender ao utilizar o *RunC*, O *LXC* simplificou a criação de sistemas *Linux* completos por meio de *templates* prontos, enquanto o *LXD* demonstrou uma forma moderna de gerenciamento de *containers*, oferecendo recursos adicionais para administração de imagens, armazenamento e configuração do ambiente. Posteriormente, o *Docker* evidenciou uma abordagem voltada à distribuição de aplicações, utilizando imagens, *Dockerfiles* e o *Docker Hub* para facilitar o desenvolvimento e a implantação de *software*. Por fim, o *Podman* apresentou uma arquitetura semelhante à do *Docker*, porém sem a necessidade de um *daemon* permanente e com suporte nativo ao modo *rootless*, reforçando aspectos relacionados à segurança.

Dessa forma, o trabalho proporcionou não apenas a aprendizagem dos comandos e procedimentos de implementação, mas também uma compreensão sólida em relação à arquitetura dos *containers* e de sua importância para ambientes modernos de desenvolvimento, computação em nuvem e infraestrutura de software.

5. REFERÊNCIAS

1 - MEDEIROS, Leonardo. Materiais de aula em PDF: “**Introdução ao Desenvolvimento Front-End com o Framework Vue.js**”. UNIOESTE, 2026.

2 - MEDEIROS, Leonardo. Materiais de aula em PDF: “**Apps Web com Vue.js**”. UNIOESTE, 2026.

3 - MEDEIROS, Leonardo. Materiais de aula: “**Design Responsivo para Web: HTML, CSS e Bootstrap**”. UNIOESTE, 2026.